Lecture 11B

Hadoop + Spark for ETL

Why we learn it, where it fits, and how to install a local lab with Vagrant

Why learn Hadoop + Spark?

- ETL moves from MB/GB files to **TB/PB** datasets quickly.
- Single-machine Python scripts become slow or memory-bound.
- Hadoop + Spark teaches **distributed thinking**:
  - partitioning
  - fault tolerance
  - cluster resource management
  - scalable batch processing
- Foundation for many data engineering roles and modern lakehouse stacks.

How it connects to our course

Topic	Role
Airflow	Orchestrates **when** pipelines run
Ollama/LLMs	Helps transform messy text to structured outputs
Hadoop (HDFS + YARN)	Distributed storage + cluster resource manager
Spark	Distributed compute engine for ETL

Pattern: Airflow DAG triggers Spark job; Spark reads/writes distributed data on HDFS/object storage.

Hadoop in 60 seconds

- **HDFS**: Distributed file system with block replication.
- **YARN**: Resource manager/scheduler for cluster jobs.
- **NameNode**: Metadata for HDFS.
- **DataNode**: Stores HDFS blocks.
- Great for reliable storage of large datasets and parallel reads.

Spark in 60 seconds

- In-memory distributed processing engine.
- APIs: **PySpark**, Scala, SQL.
- Core concepts:
  - **DataFrame** / Spark SQL
  - **Transformations** (`select` , `filter` , `join` , `groupBy`)
  - **Actions** (`count` , `write` , `collect`)
- Runs on local mode or cluster managers (YARN, Kubernetes, standalone).

ETL with Spark (typical flow)

1. **Extract** raw files / API dumps.
2. **Transform**:
    - parse schemas
    - clean nulls / bad records
    - deduplicate
    - enrich / aggregate
3. **Load** curated outputs (Parquet/Delta, warehouse, lake storage).

Spark excels in step 2 when data is too large for one machine.

Why Spark over plain Python pandas?

- pandas: excellent for local analysis, limited by one machine RAM/CPU.
- Spark: distributed execution, fault tolerance, better for large joins and aggregations.
- Unified APIs for batch + streaming + SQL.
- Better ETL scalability and production readiness.

Suggested architecture in this course

Airflow DAG:

1. Task A: fetch raw weather / source files
2. Task B: optional Ollama parsing/normalization for unstructured pieces
3. Task C: Spark ETL job (PySpark) for large-scale transformations
4. Task D: load curated outputs and quality checks

You combine **LLM-assisted parsing** with **distributed ETL**.

Installation paths (high level)

1. **Local binary mode** (fastest learning): Spark local mode, no cluster.
2. **Docker Compose** (modern quick lab): Spark + optional Hadoop services.
3. **Vagrant VM lab** (classic learning): reproducible Linux VM cluster-like setup.

Today we include a **Vagrant guide** because it mirrors old Hadoop training labs and is easy to reset.

Vagrant setup (macOS/Linux/Windows)

Prerequisites

- VirtualBox
- Vagrant
- At least 8GB RAM on host (12GB better)

Verify

```
vagrant -v
VBoxManage --version
```

Vagrant quick start (single VM lab)

```
mkdir hadoop-spark-lab && cd hadoop-spark-lab
vagrant init ubuntu/jammy64
```

Edit `Vagrantfile`:

- set RAM to 4096+ MB
- set CPUs to 2+

```
vagrant up
vagrant ssh
```

Inside VM: install Java + Hadoop + Spark (simple lab)

```
sudo apt update
sudo apt install -y openjdk-11-jdk wget curl python3-pip
java -version
```

Download Spark prebuilt for Hadoop:

```
wget https://downloads.apache.org/spark/spark-3.5.1/spark-3.5.1-bin-hadoop3.tgz
tar -xzf spark-3.5.1-bin-hadoop3.tgz
sudo mv spark-3.5.1-bin-hadoop3 /opt/spark
echo 'export PATH=/opt/spark/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
spark-submit --version
```

Minimal Spark ETL test

Create `weather_etl.py`:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("weather-etl").getOrCreate()
df = spark.read.json("raw_weather.json")
clean = df.select("latitude", "longitude", "current.temperature_2m")
clean.write.mode("overwrite").parquet("out/weather_curated")
spark.stop()
```

Run:

```
spark-submit weather_etl.py
```

Optional Hadoop layer in Vagrant lab

For full Hadoop training:

- Add Hadoop binaries (HDFS + YARN) in the VM
- Configure pseudo-distributed mode
- Start `namenode`, `datanode`, `resourcemanager`, `nodemanager`
- Run Spark on YARN (`--master yarn`)

For this course, Spark local mode is enough first; then scale to cluster mode.

Common setup issues

Problem	Fix
VM too slow / crashes	Increase RAM/CPU in `Vagrantfile`
`JAVA_HOME` errors	Set `JAVA_HOME` correctly and reload shell
`spark-submit` not found	Add `/opt/spark/bin` to PATH
Port conflicts	Forward different ports in `Vagrantfile`
Airflow cannot call Spark	Use SSH/BashOperator or SparkSubmitOperator with reachable binaries

Assignment (new)

1. Bring up Vagrant VM and install Spark.
2. Fetch weather JSON (Open-Meteo) as raw input.
3. Run Spark ETL that outputs a curated table/file.
4. (Bonus) Add Airflow task before Spark to use Ollama for text normalization.
5. Submit screenshots + code + short architecture note.

Summary

- Hadoop + Spark are core for **distributed ETL**.
- We learn them to move from local scripts to scalable pipelines.
- Airflow orchestrates jobs; Spark transforms large datasets.
- Vagrant gives a reproducible local lab to practice setup and operations.

Next: implement the assignment pipeline end-to-end.